

// Jivops

OpenEBS

Guía completa — 20 lecciones

Fundamentos · Almacenamiento local · Replicated PV (Mayastor) · Operación y producción

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es OpenEBS y el modelo CAS
- Lección 02 — Motores de almacenamiento y cuál elegir
- Lección 03 — Instalación con Helm
- Lección 04 — StorageClass y volumeBindingMode
- Lección 05 — PV, PVC y política de reclamación

Módulo 2: Almacenamiento local

- Lección 06 — Local PV Hostpath
- Lección 07 — Local PV LVM
- Lección 08 — Local PV ZFS
- Lección 09 — Snapshots y clones
- Lección 10 — Expansión de volúmenes y gestión de capacidad

Módulo 3: Replicated PV (Mayastor)

- Lección 11 — Arquitectura de Replicated PV (Mayastor)
- Lección 12 — Requisitos de Mayastor
- Lección 13 — DiskPools
- Lección 14 — Réplicas y alta disponibilidad
- Lección 15 — Rendimiento y ajuste

Módulo 4: Operación y producción

- Lección 16 — Bases de datos y StatefulSets
- Lección 17 — Monitorización
- Lección 18 — Backup y restauración
- Lección 19 — Actualizaciones y troubleshooting
- Lección 20 — Proyecto final: OpenEBS listo para producción

Módulo 1: Fundamentos

Lección 01 — Qué es OpenEBS y el modelo CAS

Una solución de almacenamiento nativa de Kubernetes (proyecto de la CNCF) que ofrece volúmenes persistentes mediante drivers CSI y el modelo Container Attached Storage, donde el almacenamiento se gestiona con componentes que corren como pods del propio cluster.

Que sus componentes corren como pods del propio cluster y se gestionan con objetos de Kubernetes (StorageClass, PVC) a través de drivers CSI, sin una cabina externa.

Lección 02 — Motores de almacenamiento y cuál elegir

Según el requisito: los motores Local PV (Hostpath, LVM, ZFS) dan volúmenes ligados a un nodo con poca sobrecarga y sin replicación, y Replicated PV (Mayastor) replica los datos entre nodos para alta disponibilidad a costa de más recursos y requisitos.

Un Local PV: la propia base de datos ya replica, así que no hace falta replicar también el almacenamiento y se evita la sobrecarga adicional.

Lección 03 — Instalación con Helm

Con el chart de Helm openebs/openebs en el namespace openebs, donde se activan o desactivan los motores según se necesiten (por ejemplo LVM o Replicated PV), y comprobando después que los pods estén en Running y que existan las StorageClass.

Que los pods del namespace openebs estén en Running y que se hayan creado las StorageClass de los motores activados.

Lección 04 — StorageClass y volumeBindingMode

Porque retrasa la creación del volumen hasta que un pod que lo usa es programado, de modo que el volumen se crea en el nodo elegido por el scheduler y no en uno distinto donde el pod no podría usarlo.

Evita que el volumen se cree en un nodo y el pod se programe en otro: el volumen se crea donde el scheduler coloca el pod.

Lección 05 — PV, PVC y política de reclamación

Qué ocurre con el volumen al borrar el PVC: con Delete se elimina también el PV y sus datos, y con Retain se conserva el PV y los datos para recuperarlos manualmente.

Para evitar perder datos ante un borrado accidental del PVC: el PV y su contenido se conservan y pueden recuperarse manualmente.

Módulo 2: Almacenamiento local

Lección 06 — Local PV Hostpath

Crea cada volumen como un directorio en el sistema de ficheros del nodo (ruta base configurable, por defecto `/var/openebs/local`), sin replicación, de modo que el pod queda ligado a ese nodo y los datos se pierden si el nodo o su disco fallan.

Los datos viven en un solo nodo sin replicación: si falla se pierden; es aceptable para cachés, desarrollo o aplicaciones que ya replican sus datos.

Lección 07 — Local PV LVM

Crea volúmenes lógicos LVM sobre un grupo de volúmenes (volume group) existente en el nodo, lo que añade capacidad de reservar tamaño, ampliar volúmenes y hacer snapshots, aunque sigue sin replicar y requiere preparar LVM en los nodos.

Instalar las herramientas de LVM y crear un volume group sobre un disco del nodo, cuyo nombre se indica en la StorageClass.

Lección 08 — Local PV ZFS

Para crear volúmenes sobre un pool ZFS del nodo aprovechando sus funciones, como snapshots, clones, compresión y suma de verificación de datos; igual que los otros Local PV no replica entre nodos y exige el módulo ZFS y un pool en cada nodo.

Snapshots y clones eficientes, compresión y comprobación de integridad de los datos, aunque sin replicación entre nodos.

Lección 09 — Snapshots y clones

Que el motor lo soporte (por ejemplo LVM o ZFS), que estén instalados los CRD de VolumeSnapshot y el controlador de snapshots del cluster y que exista una VolumeSnapshotClass con el driver correspondiente para crear objetos VolumeSnapshot.

Creando un PVC nuevo cuyo dataSource apunta al VolumeSnapshot; el driver crea el volumen a partir de la instantánea.

Lección 10 — Expansión de volúmenes y gestión de capacidad

Que la StorageClass tenga `allowVolumeExpansion: true` y que el motor lo soporte; entonces se edita el PVC con un tamaño mayor y el driver amplía el volumen, siendo importante vigilar también la capacidad libre del nodo o del pool.

Porque si se asigna más de lo que hay, el espacio del nodo o del pool puede agotarse y fallan las escrituras; hay que vigilar la capacidad libre.

Módulo 3: Replicated PV (Mayastor)

Lección 11 — Arquitectura de Replicated PV (Mayastor)

Un plano de control (agentes y controlador CSI que gestionan volúmenes, pools y réplicas) y un plano de datos formado por el io-engine, que corre en los nodos de almacenamiento y expone los volúmenes por NVMe-oF, replicando las escrituras en las réplicas.

El control gestiona pools, volúmenes y réplicas, y el plano de datos (io-engine) sirve las E/S; así un fallo del control no detiene necesariamente las E/S activas.

Lección 12 — Requisitos de Mayastor

Hugepages configuradas (por ejemplo 2 GiB de páginas de 2 MiB), el módulo de kernel `nvme_tcp`, CPU reservada para el io-engine, que usa núcleos de forma intensiva, y discos dedicados para los pools de los nodos de almacenamiento.

Porque el io-engine reserva hugepages y núcleos de CPU de forma dedicada, y en nodos pequeños esto compite con las cargas de trabajo.

Lección 13 — DiskPools

Un recurso que agrupa uno o varios discos de un nodo concreto y es la unidad de capacidad desde la que se crean las réplicas de los volúmenes; se declara con un objeto `DiskPool` indicando el nodo y los discos.

Porque cada réplica se coloca en un pool de un nodo distinto, así el volumen tolera la caída de un nodo.

Lección 14 — Réplicas y alta disponibilidad

Con el parámetro `repl` de la `StorageClass` (por ejemplo `repl: "3"`) se define cuántas copias se mantienen en nodos distintos; con réplicas suficientes el volumen sigue disponible si cae un nodo y la réplica perdida se reconstruye.

Más réplicas dan más disponibilidad pero consumen más capacidad y pueden aumentar la latencia de escritura, porque cada escritura debe replicarse.

Lección 15 — Rendimiento y ajuste

La red entre nodos (la replicación viaja por NVMe-oF/TCP), el tipo de disco (NVMe frente a HDD), los núcleos asignados al io-engine y el número de réplicas, por lo que conviene medir con una herramienta como `fio` antes de producción.

Ejecutando pruebas con `fio` sobre un PVC real, midiendo IOPS y latencia con el mismo patrón de acceso que la base de datos.

Módulo 4: Operación y producción

Lección 16 — Bases de datos y StatefulSets

Como StatefulSet con volumeClaimTemplates, usando Local PV si la base de datos ya replica por sí misma (con anti-afinidad para repartir pods en nodos distintos) o Replicated PV si se necesita que el almacenamiento sobreviva a la pérdida de un nodo.

Para que cada réplica quede en un nodo distinto: así un fallo de nodo solo afecta a una réplica de la base de datos.

Lección 17 — Monitorización

Con Prometheus y Grafana: los componentes exponen métricas (por ejemplo de pools y volúmenes en Mayastor), se vigilan la capacidad libre, el estado de las réplicas y la latencia, y se configuran alertas sobre pools casi llenos o volúmenes degradados.

Pool o disco casi lleno (poca capacidad libre) y volumen degradado (réplicas caídas o en reconstrucción).

Lección 18 — Backup y restauración

Porque las réplicas copian también los borrados y la corrupción lógica, y un fallo de todo el cluster o un error humano afecta a todas; el backup (por ejemplo con snapshots enviados fuera con Velero) es una copia independiente en el tiempo y en otra ubicación.

No: las réplicas también reciben el borrado, así que hay que restaurar desde un backup o snapshot anterior.

Lección 19 — Actualizaciones y troubleshooting

Revisando con kubectl describe los eventos del PVC y del pod, comprobando que existe la StorageClass indicada, que los pods del driver están en Running, que hay capacidad en el nodo o pool y que el nodo del pod coincide con el volumen local.

Leer las notas de la versión, tener un backup reciente y probar la actualización en un entorno de pruebas antes de producción.

Lección 20 — Proyecto final: OpenEBS listo para producción

Elegir el motor según la carga (Local PV para datos ya replicados por la aplicación y Replicated PV con repl 3 para el resto), con StorageClass WaitForFirstConsumer, reclaimPolicy Retain, monitorización con alertas de capacidad y backups probados fuera del cluster.

Motor adecuado a la carga (con replicación si hace falta), monitorización con alertas de capacidad y backups restaurados con éxito en una prueba.